

北京邮电大学

《软件安全》



题目： 模糊测试技术概述及实验研究

学 号： 2022211570

姓 名： 项 枫

班 级： 2022211801

专 业： 信息安全

学 院： 网络空间安全学院

2024 年 11 月 13 日

目 录

一、模糊测试技术概述	1
(一) 发展历史	1
(二) 模糊测试技术方法进展	2
1、面向种子选择的能量调度的改进	2
2、面向测试用例变异算法的改进	3
3、面向模糊测试执行性能的改进	3
4、基于混合模糊测试的改进	4
(三) 模糊测试技术应用进展	4
1、软件测试	5
2、协议测试	6
3、其他测试	7
(四) 模糊测试技术的挑战与机遇	8
1、反模糊测试	8
2、模糊工具的集成	8
3、深度学习框架的模糊测试	8
二、关于 Golang 的模糊测试实践	8
(一) 对 Go 字符串反转函数进行模糊测试	9
1、代码编写	9
2、运行测试	11
3、运行结果	11
(二) 在 Go 中验证和存储 CSV 数据：模糊测试方法	12
1、代码编写	12
2、运行测试	15
3、运行结果	15
(三) 实验结论	16
1、模糊测试的有效性	16
2、模糊测试的局限性	16
(四) 实验心得	17
1、模糊测试的实用性	17

2、代码设计的重要性	17
3、对边界情况的重视	17
4、未来的改进方向	17
三、关于模糊测试技术的思考与创新性观点	17
（一）融合人工智能与机器学习	18
（二）基于形式化方法的模糊测试	18
（三）增强动态反馈机制	18
（四）提升对未知协议和系统的适应性	18
（五）实现跨平台的模糊测试框架	19
（六）社区协作与开源创新	19
参考文献	20
致 谢	21

一、模糊测试技术概述

模糊测试技术是一种自动化的漏洞挖掘方法,通过向被测程序输入各种无效、不合法或随机的数据来发现程序潜在的异常情况^[4]。具体来说,是基于对目标测试对象的分析,使用种子变异算法对正常输入数据进行变异,以伪随机的方式构造大量预期或非预期的测试用例,作为输入传递给目标测试对象,来发现意外的程序行为,并通过分析目标的输出结果进行漏洞检测的方法。在模糊测试的过程中,大部分阶段都是自动化实现的,无需较多的专业人员经验即可进行,具有测试效率高、速度快的优点,同时,由于模糊测试是在程序运行过程中动态监控的,所以误报率较低,而自动化的测试用例生成策略也有利于发现未知的漏洞,提高了软件漏洞发现的概率^[2]。

（一）发展历史

如图 1 所示,模糊测试技术是软件安全领域中的一项重要技术,其发展历史已有约 30 年。

模糊测试诞生于 1988 年,由 Miller 等人^[1]率先提出,并在那时设计了一个被称为 Fuzz 的工具,通过生成随机连续字符串对以字符串为输入的对象进行模糊测试,同时设计了一个名为 `ptyjig` 的工具,对输入有特殊要求的目标进行模糊测试。这时的模糊测试,其主要目的是尝试使用非常规的数据对目标的鲁棒性进行检测^[5]。

2001 年,Protos 项目诞生,标志着模糊测试应用于网络协议分析的新起点,也是模糊测试技术成为实用性工具的开始。2002 年,Aitel 详细讲解并演示了基于生成的模糊测试工具 SPIKE。2008 年,针对 Microsoft Office 等复合文档安全,Gao 等人提出了一种可以有效提高测试效率的测试用例构造方法。2009 年,基于污点分析的 Vganesh 技术被广泛应用。2010 年,Wang 等人结合混合符号执行与细粒度动态污点跟踪技术,提出一种绕过系统校验和防护机制的方法,提高了漏洞挖掘能力。2011 年,Zhu 等人提出了模糊测试组件生成模型,可以从庞大的数据集样本中自动生成具有较高覆盖率的测试用例^[6]。在 2013 年,AFL 的出现

推动了模糊测试技术的再次进步，成为了模糊测试工具和技术的重要里程碑之一。AFL 工具引入了许多创新的技术，例如基于覆盖率引导的反馈机制和高效的变异策略，使得模糊测试的效率得到大幅度的提升。同时，人工智能算法的应用让模糊测试在智能化程度和测试效率上取得了新突破。这些技术有效提高了整体模糊测试的代码覆盖率，从而更全面地检测目标程序中的潜在漏洞和问题^[3]。

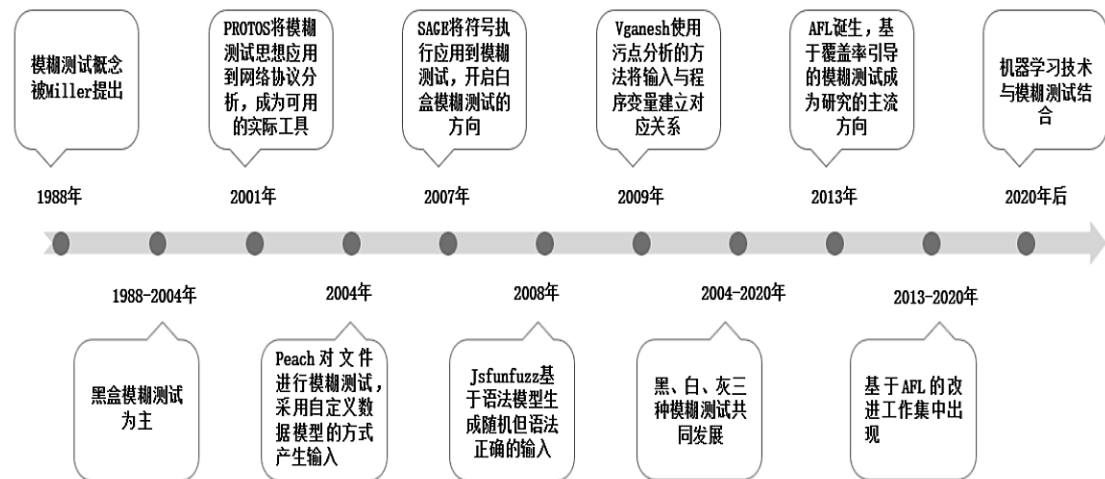


图 1 模糊测试技术发展历史

（二）模糊测试技术方法进展^[2]

1、面向种子选择的能量调度的改进

在模糊测试工具 AFL 中，每个种子的变异次数基本是一样的，但是实际上不同的种子能够覆盖的代码分支的数量和频次都有所差别，进而可能影响变异生成的测试用例的边覆盖效果。因此，研究者们从种子的选择及其变异次数的角度对 AFL 进行改进，根据各个种子的特点采取相应的能量分配策略，控制每个种子执行的最优变异次数，以提高模糊测试的漏洞挖掘效率。

Böhme 等人提出了 AFLFast，将基于覆盖的灰盒模糊测试建模为对马尔科夫链的状态空间的系统探索，通过偏向低频路径的能量分配策略，以在同等时间内执行更多的低频路径，增加了模糊测试覆盖更多路径的可能性。同年，Böhme 等人面向定向模糊测试场景提出了 AFLGo，基于模拟退火算法，以给定的源码位

置为目标生成输入，将更多的能量分配给离目标位置较近的种子，同时降低离目标位置较远的种子的能量，从而达到更快覆盖给定位置的源码的效果。

为了增加 AFL 所能覆盖的代码区域，Lemieux 等人提出了 FairFuzz，通过采用偏向产生稀有分支的种子能量分配策略，控制种子的变异次数，以提高模糊测试的覆盖率。由于准确的路径覆盖需要较高的成本，大多数模糊测试通常使用粗略的覆盖信息，种子选择策略主要聚集在路径访问频次、路径深度、执行速度等方面，为了解决这一问题，Gan 等人提出了 CollAFL，通过基于敏感路径的种子能量分配策略，将模糊测试引导到未探索的相邻分支路径、相邻后续路径以及存在较多内存访问操作的路径，从而提供更准确的路径覆盖信息减轻路径冲突，提高发现新路径和新漏洞的效率。

针对 AFLFast 的马尔科夫链建模不够深刻，以致能量调度比较单一的问题，Yue 等人提出了 EcoFuzz，使用对抗性多臂老虎机的变体(VAMAB)对基于覆盖的灰盒模糊测试进行建模，通过基于种子执行次数与发现新路径的平均成本的能量分配策略，实现了种子能量分配的动态调整，从而降低了模糊测试的能量消耗和平均成本。

2、面向测试用例变异算法的改进

测试用例生成是整个模糊测试过程中的关键步骤，变异算法决定了生成数据的有效性，可直接影响模糊测试的覆盖率以及漏洞发现效率。因此，面向测试用例变异算法的改进受到了研究者的广泛关注。

针对结构化输入的目标程序，Wang 等人提出了基于语法感知的模糊测试技术 Superion，通过抽象语法树解析测试输入，以及在树的层面直接修剪，并基于字典和基于树的 2 种语法感知的变异策略，可以生成语法语义有效的测试用例。为了在有限的时间内高效快速地发现二进制程序中的漏洞，Li 等人结合脆弱性预测和进化算法提出了 V-Fuzz，通过基于图神经网络的漏洞预测模型，预估程序中更容易受到攻击的部分，并利用进化算法生成更有可能到达漏洞位置的测试用例，提高了模糊测试的速度。

3、面向模糊测试执行性能的改进

模糊测试的执行性能可以直观地展示模糊测试工具的效率，由于任务调度、系统可扩展性、代码插桩、测试用例追踪等诸多因素的影响，模糊测试的资源成本、执行时间、挖掘效率也相应地受到一些限制，因此，对于这方面的改进还有较多的发展空间，以下是一些面向模糊测试执行性能方面的研究。

为了减少 AFL 追踪测试用例覆盖率产生的性能开销，Nagy 等人提出了 UnTracer，通过量化测试用例与代码覆盖率的频率之间的关系，筛选出能够增加覆盖率的测试用例，并对其进行追踪，同时减少对不能增加覆盖率的测试用例的追踪，进而降低基于覆盖引导的模糊测试的开销。为了节省模糊处理的资源成本、提高模糊测试效率，Zhou 等人提出了一种分布式模糊测试方法 UltraFuzz，通过基于集中式的动态调度和弹性分配计算能力，实现种子能量调度的全局优化，解决了并行模式下，由于缺乏合适的任务调度方案和实时状态同步导致的任务冲突、负载不均、资源浪费等问题。

4、基于混合模糊测试的改进

混合模糊测试一般指结合模糊测试与符号执行的技术，利用模糊测试来测试容易到达的代码区域，并使用符号执行探索由复杂分支条件保护的代码块，通过 2 种技术的结合使用达到挖掘更深的程序状态空间的效果。但是混合测试也面临生成约束慢、路径爆炸、需要源码以及无效测试用例等带来的性能瓶颈，虽然混合模糊测试起步较晚，但是随着研究的不断深入，近年来也取得一些进展。

Stephens 等人提出了 Driller，结合动态符号执行解决了模糊测试对本块校验检查的不足，能实现选择性地探索感兴趣的程序路径，并帮助模糊测试跳过不能满足的判断条件语句路径，能够探索更深层次的漏洞，避免了符号执行的路径爆炸和模糊测试的不完全性问题。不足之处在于较为损耗计算资源，且为白盒测试，需要源代码。针对模糊测试挖掘效率低、程序状态遍历复杂的问题，Zhao 等人提出了概率路径优先的混合测试工具 DigFuzz，通过基于蒙特卡洛搜索的概率路径优先级模型，量化评估每条路径的探索难度并排序，然后利用符号执行探索困难路径，大幅度提升了漏洞挖掘的效率和性能。

（三）模糊测试技术应用进展^[6]

1、软件测试

(1) Android 测试

随着技术水平和社会经济的迅速发展,智能手机在人们的日常生活中越来越普及,从衣食住行等不同方面影响着人们的生产和生活方式。智能手机开发平台多种多样,其中 Android 是应用最为广泛的平台之一。为应对快速增长的系统安全威胁,研究人员提出了针对智能手机系统安全的解决方法,例如静态分析技术和自动动态分析技术。

有效触发恶意行为是实现智能手机恶意软件自动化分析的前提。Wang 等人专注于对抗 Android 恶意软件中的各种躲避技术,在自动动态分析中触发更多隐藏的恶意行为,从而帮助生成更全面的恶意软件报告。针对 Android 组件通信过程中由于组件暴露产生的安全问题,王凯等人结合模糊测试和逆向分析技术,设计并实现了 Android 漏洞挖掘工具 KMDroid。张密等人提出了一种基于模糊测试方法的组件通信测试方案,以 Android SDK 提供的数据为基础,引导高覆盖率的测试用例生成,设计并实现了工具 FuzzerAPP。关于 Android 应用程序的健壮性测试,赛 等提出了一种基于模糊测试的检测方法,使用 Android 组件相关信息来构造测试用例,设计实现了全自动测试工具 ICCDroidFuzzer,实验中结合测试结果对源码进行分析,得出了抛出异常的根本原因。针对 Android 驱动安全,何远等人提出了基于黑盒模糊测试的遗传算法,通过测试执行结果的实时反馈来引导遗传算法,实现了 Android 驱动的模糊测试系统 Add-fuzz(Android device driverfuzz)。

(2) Windows 测试

图形用户界面 GUI(Graphical User Interface)可视化计算机程序,其目的是促进用户和设备之间的交互。GUI 测试是软件测试的关键部分,验证界面和用户之间的交互行为,而不考虑任何编码细节。Din 等人提出了一种基于模糊自适应教与学优化 ATLBO(Adaptive Teaching Learning-Based Optimization)算法的 GUI 功能测试策略,该算法是基于教与学优化 TLBO(Teaching Learning-Based Optimization)算法的变体,利用事件交互图 EIG(Event-Interaction Graph)来生成高

质量测试用例。为提高 GUI 测试时空转时机判定的准确率，张兴等人提出了基于 BiGram 模型以及统计分析的空转状态识别方法。

（3）浏览器测试

针对浏览器安全，Zalewski 提出了模糊测试工具 AFL(AmericanFuzzy Lop)，根据测试结果计算覆盖率，进而指导数据变异生成测试用例。霍玮等提出了一种基于模式生成的浏览器模糊测试方法，通过对测试模式分析、定义与自动化提取以产生模糊测试器完成测试数据构造，设计并实现了一个浏览器模糊测试工具 autofuzzy。不足之处在于对获取的测试模式没有一个较好的管理和反馈机制，还需要更多的基础数据来满足样本的生成。

2、协议测试

（1）互联网协议

网络协议的安全问题近期受到人们的广泛关注。随着网络协议逆向工程的提出，针对手工模糊测试操作繁琐且代码覆盖率低的问题，李伟明等人采用了一种可以对网络协议实现自动化识别并且能对其进行有效模糊测试的漏洞挖掘方法，显著提升了测试效率。针对现有网络不能较好地支持状态转换的问题，Ma 等人根据有状态网络协议的特点，提出了有状态网络协议半有效模糊测试 SFSNP(Semi-valid Fuzzing for the Stateful Network Protocol)，通过分析协议交互，构建出带有路径标记的扩展有限状态机，并根据扩展有限状态机、协议状态规则树及半有效算法获得模糊测试序列，采用状态转换标记算法，以减少冗余测试用例生成，缩短测试时间。针对传输层安全协议 TLS(Transport Layer Security)安全性测试，Walz 等人设计了一种通用的动态数据结构——通用消息树 GMTs(Generic Message Trees)。基于 GMT 概念，提出了一种随机算法来生成高度多样化和大部分有效的 TLS 握手消息；使用消息生成算法来差异测试现有较为流行的 TLS 服务器，经过实验验证其算法的高效性；开发的工具 tls-diff-testing 作为开源项目已经向外界公布。

支持实时流传输协议 RTSP(Real Time Streaming Protocol)的视频监控设备在市场中应用广泛，针对 RTSP 协议模糊测试用例改进，Ma 等人先后提出了基于

分类树、启发式算子、规则状态机和状态树指导模糊测试用例生成的方法，可有效删除无用项，大大提高了测试数据生成质量。

（2）工控协议

工业控制系统是用于管理关键基础设施服务的操作技术系统的运行和功能的重要组成部分。Tacliad 等人开发了一款模糊测试工具 ENIP Fuzz，用于检测可编程逻辑控制器中使用的以太网/IP 软件漏洞。针对传统模糊测试应用于数据采集与监控系统 SCADA(Supervisory Control And Data Acquisition)的不足，张亚丰等人提出了一种基于状态的工控协议模糊测试方法，设计基于协议状态机的测试序列生成算法 PSTSGM(Protocol State based Test Sequences Generating Method)，提出了基于心跳的异常监测与定位方法 HFDLM(Heartbeat based Faults Detection and Location Method)，设计并实现了原型系统 SCADA-Fuzz，在此基础上对实际电力 SCADA 系统进行了测试，得到了良好的实验效果。为提高模糊测试用例的覆盖率和模糊测试效率，针对工控协议制造报文规范 MMS(Manufacturing Message Specification)的模糊测试，Kim 等人采用对数据相关信息进行分类的方法引导测试用例生成。Li 等人引入遗传算法并设计了其适应度函数来生成测试用例。Wang 等人采用静态分析、动态监测和符号执行等策略，选择出安全性更强的种子对测试对象进行定向模糊测试，设计并实现了 SeededFuzz 模型。李佳莉等人提出了根据状态关系构造协议状态图，并基于协议状态图进行深度遍历选择覆盖率更高的测试用例的方法。

（3）未知协议

随着互联网技术的不断进步，大量未知的、私有的通信协议广泛应用在不同领域。针对未知协议的复杂多样且安全性较差的问题，张蔚瑶等人提出基于协议特征库的未知协议的逆向分析方法，自动学习协议结构和语义特征，提出多维变异的测试数据自动化生成方法生成测试数据，设计并实现了自动化模糊测试工具 UPAFuzz。

3、其他测试

除了上述应用场景外，模糊测试还广泛应用在内核安全测试、机器人操作系统 ROS(Robot Operating System)、云计算、嵌入式固件、软件定义网络 SDN(Software Defined Network)和大数等方面。

(四) 模糊测试技术的挑战与机遇

1、反模糊测试

目前主流的模糊测试技术通常依赖于 4 个前提：1) 单次执行速度要足够快；2) 模糊工具可以获得覆盖率的反馈；3) 目标程序中的路径约束可以被符号求解；4) 崩溃可以被模糊工具所检测到。

基于这些依赖条件，为了实现反模糊测试，要在不对正常执行造成过大的影响的条件下，减缓模糊测试的执行速度，或者通过干扰覆盖率图，使得模糊工具无法从覆盖率图中获取有效信息，也可以干扰混合执行，使得正常能够被符号执行或污点分析求解的内容变得无法求解^[5]。

反模糊测试技术一方面可以阻止恶意模糊测试对安全的威胁，另一方面也给今后的模糊测试技术提出了新的挑战。

2、模糊工具的集成

研究人员针对不同的领域提出不同的模糊测试方法，产生了种类繁多的模糊测试工具。研发出一种适用于所有场景的模糊工具并不现实，如何将不同模糊测试工具进行整合，构造一个通用模糊测试平台，是模糊测试实用化的一个重要研究方向^[6]。

3、深度学习框架的模糊测试

随着深度学习技术在多个领域的广泛应用，其框架的安全性和稳定性也变得尤为重要。针对模型库、深度学习框架及编译器的模糊测试极为重要，如模型变异、权重生成、样例构造和模型测试等关键技术，未来的研究的主要方向为错误定位与自动修复技术、大语言模型增强的模糊测试^[7]。

二、关于 Golang 的模糊测试实践

Go 的出现让模糊测试变得轻而易举。由于工具链内置了支持，Go 开发人员可以轻松的将这种强大的测试方法自动化。这就像为代码配备了时刻保持警惕的守护者，不断查找那些可能会漏掉的偷偷摸摸的 bug。Go 模糊测试就是要将代码推向极限，甚至超越极限，以确保代码在现实世界中能够抵御任何奇特而美妙的输入。这证明了 Go 对可靠性和安全性的承诺，在一个软件需要坚如磐石的世界里，它能让人高枕无忧。因此，如果你发现应用程序即使在最意想不到的情况下也能流畅运行时，请记住模糊测试所发挥的作用，它作为无名英雄在幕后为 Go 应用的顺利运行而努力。

种子语料库(Seed Corpus)是提供给模糊测试流程的初始输入集合，用于启动生成测试用例，可以把它想象成锁匠用来制作万能钥匙的初始钥匙集。在模糊测试中，这些种子作为起点，模糊器从中衍生出多种变体，探索大量可能的输入以发现错误。通过精心挑选一组具有代表性的多样化种子，可以确保模糊器从一开始就能覆盖更多领域，从而使测试过程更高效且有效。种子可以是典型用例数据，也可以是边缘用例或以前发现的可诱发错误的输入，从而为彻底测试软件的可靠性奠定基础。

（一）对 Go 字符串反转函数进行模糊测试^[8]

我们用 Go 编写一个简单的字符串反转函数，然后创建一个模糊测试。

完整工程文件请见文件夹“Fuzzing_Go_ReverseStr”。

1、代码编写

（1）Go 函数：反转字符串(Reverse.go)

```
package main

// ReverseString takes a string as input and returns its reverse.
func ReverseString(s string) string {
    // Convert the string to a rune slice to properly handle multi-byte characters.
    runes := []rune(s)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        // Swap the runes.
        runes[i], runes[j] = runes[j], runes[i]
    }
}
```

```

}
// Convert the rune slice back to a string and return it.
return string(runes)
}

```

ReverseString 函数：该函数接收一个字符串参数并返回其反转值。它将字符串作为 `rune` 切片而不是字节来处理，这对于正确处理大小可能超过一个字节的 Unicode 字符至关重要。

Rune 切片：通过将字符串转换为 `rune` 切片，可以确保正确处理多字节字符，保持字符编码的完整性。

交换：该函数迭代 `rune` 切片，从两端开始交换元素，然后向中心移动，从而有效反转切片。

(2) ReverseString 函数的模糊测试(reverse_fuzz_test.go)

```

package main

import (
    "testing"    "unicode/utf8"
)

// FuzzReverseString tests the ReverseString function with fuzzing.
func FuzzReverseString(f *testing.F) {
    f.Add("hello")
    f.Add("world")
    f.Add("你好") // "Hello" in Chinese

    f.Fuzz(func(t *testing.T, original string) {
        // Check if the original string is valid UTF-8
        if !utf8.ValidString(original) {
            t.Skip("Skipping invalid UTF-8 string")
        }

        reversed := ReverseString(original)
        doubleReversed := ReverseString(reversed)
        if original != doubleReversed {
            t.Errorf("Double reversing '%s' did not give original string, got '%s'",
                original, doubleReversed)
        }
    })
}

```

```

        if utf8.RuneCountInString(original) != utf8.RuneCountInString(reversed) {
            t.Errorf("The length of the original and reversed string does not match for
            '%s'", original)
        }
    })
}

```

种子语料库：我们从一组种子输入开始，包括简单的 ASCII 字符串和一个 Unicode 字符串，以确保模糊测试涵盖一系列字符编码。

模糊函数：模糊函数反转字符串，然后再反转回来，期望得到原始字符串。这是一个简单的不变量，如果反转函数正确的话，就应该总是成立的。它还会检查原始字符串和反转字符串的长度是否相同，以检查多字节字符可能出现的问题。

2、运行测试

使用以下命令：

```
go test -fuzz=Fuzz -fuzztime=60s
```

3、运行结果

```

fuzz: elapsed: 0s, gathering baseline coverage: 0/43 completed

fuzz: elapsed: 0s, gathering baseline coverage: 43/43 completed, now fuzzing with 8
workers

fuzz: elapsed: 3s, execs: 109848 (36609/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 6s, execs: 125064 (5046/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 9s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 12s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 15s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 18s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 21s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 24s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 27s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 30s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 33s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 36s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 39s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 42s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 45s, execs: 125064 (0/sec), new interesting: 1 (total: 44)

```

```
fuzz: elapsed: 48s, execs: 2273435 (719018/sec), new interesting: 2 (total: 45)
fuzz: elapsed: 51s, execs: 4080676 (603167/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 54s, execs: 4873379 (263350/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 57s, execs: 5171268 (99582/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 1m0s, execs: 5463205 (97492/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 1m0s, execs: 5464633 (19529/sec), new interesting: 3 (total: 46)
PASS
ok      Fuzzing_of_Go    60.375s
```

```
PS D:\SoftwareSecurity\homework\hw2\Fuzzing_Go_ReverseStr> go test -fuzz=Fuzz -fuzztime=60s
fuzz: elapsed: 0s, gathering baseline coverage: 0/43 completed
fuzz: elapsed: 0s, gathering baseline coverage: 43/43 completed, now fuzzing with 8 workers
fuzz: elapsed: 3s, execs: 109848 (36609/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 6s, execs: 125064 (5046/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 9s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 12s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 15s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 18s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 21s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 24s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 27s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 30s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 33s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 36s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 39s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 42s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 45s, execs: 125064 (0/sec), new interesting: 1 (total: 44)
fuzz: elapsed: 48s, execs: 2273435 (719018/sec), new interesting: 2 (total: 45)
fuzz: elapsed: 51s, execs: 4080676 (603167/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 54s, execs: 4873379 (263350/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 57s, execs: 5171268 (99582/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 1m0s, execs: 5463205 (97492/sec), new interesting: 3 (total: 46)
fuzz: elapsed: 1m0s, execs: 5464633 (19529/sec), new interesting: 3 (total: 46)
PASS
ok      Fuzzing_of_Go    60.375s
```

图 2 对 Go 字符串反转函数进行模糊测试结果截图

模糊测试运行了 60 秒，执行了超过 5,460,000 次函数调用，并发现了 3 个新的“interesting”输入，使得总数达到了 46 个。

（二）在 Go 中验证和存储 CSV 数据：模糊测试方法^[8]

要创建一个读取 CSV 文件、验证其值并将验证后的数据存储到文件中的函数，我们将按照以下步骤进行操作，1）处理 CSV 的函数，该函数将读取 CSV 数据，根据预定义规则验证其内容（为简单起见，假设我们期望两列具有特定的数据类型），然后将验证后的数据存储到新文件中；2）模糊测试：我们将为验证 CSV 数据的函数部分编写模糊测试，这是因为模糊测试非常适合测试代码如何处理各种输入，而我们将重点关注验证逻辑。

完整工程文件请见文件夹“Fuzzing_Go_CSV”。

1、代码编写

（1）处理和验证 CSV 数据(csv.go)

```
package main

import (
    "encoding/csv"    "fmt"    "io"    "os"    "strconv"
)

// validateAndSaveData reads CSV data from an io.Reader, validates it, and saves valid
rows to a file.
func validateAndSaveData(r io.Reader, outputFile string) error {
    csvReader := csv.NewReader(r)
    validData := [][]string{}

    for {
        record, err := csvReader.Read()
        if err == io.EOF {
            break
        }
        if err != nil {
            return fmt.Errorf("error reading CSV data: %w", err)
        }

        if validateRecord(record) {
            validData = append(validData, record)
        }
    }

    return saveValidData(validData, outputFile)
}

// validateRecord checks if a CSV record is valid. For simplicity, let's assume the
first column should be an integer and the second a non-empty string.
func validateRecord(record []string) bool {
    if len(record) != 2 {
        return false
    }

    if _, err := strconv.Atoi(record[0]); err != nil {
        return false
    }

    if record[1] == "" {
        return false
    }
}
```



```

    return true
}

// saveValidData writes the validated data to a file.
func saveValidData(data [][]string, outputFile string) error {
    file, err := os.Create(outputFile)
    if err != nil {
        return fmt.Errorf("error creating output file: %w", err)
    }
    defer file.Close()

    csvWriter := csv.NewWriter(file)
    for _, record := range data {
        if err := csvWriter.Write(record); err != nil {
            return fmt.Errorf("error writing record to file: %w", err)
        }
    }
    csvWriter.Flush()
    return csvWriter.Error()
}

```

`validateAndSaveData` 函数从 `io.Reader` 读取数据，从而可以处理来自任何实现此接口的数据源（如文件或内存缓冲区）的数据。该函数通过 `csv.Reader` 解析 CSV 数据，使用 `validateRecord` 验证每条记录，并存储有效记录。

`validateRecord` 函数旨在根据简单的规则验证每条 CSV 记录：第一列必须可转换为整数，第二列必须是非空字符串。

`saveValidData` 函数获取经过验证的数据，并以 CSV 格式将其写入指定的输出文件。

（2）验证逻辑的模糊测试(csv_fuzz_test.go)

```

package main

import (
    "strings"    "testing"
)

// FuzzValidateRecord tests the validateRecord function with fuzzing.

```

```
func FuzzValidateRecord(f *testing.F) {
    // Seed corpus with examples, joined as single strings
    f.Add("123,validString") // valid record
    f.Add("invalidInt,string") // invalid integer
    f.Add("123,") // invalid string

    f.Fuzz(func(t *testing.T, recordStr string) {
        // Split the string back into a slice
        record := strings.Split(recordStr, ",")

        // Now you can call validateRecord with the slice
        _ = validateRecord(record)
        // Here you can add checks to verify the behavior of validateRecord
    })
}
```

FuzzValidateRecord 函数的模糊测试使用种子输入来启动模糊处理过程，用大量生成的输入值来测试验证逻辑，以发现潜在的边缘情况或意外行为。

2、运行测试

使用以下命令：

```
go test -fuzz=Fuzz -fuzztime=60s
```

3、运行结果

```
fuzz: elapsed: 0s, gathering baseline coverage: 0/34 completed

fuzz: elapsed: 0s, gathering baseline coverage: 34/34 completed, now fuzzing with 8
workers

fuzz: elapsed: 3s, execs: 398470 (132417/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 6s, execs: 837395 (146362/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 9s, execs: 1287657 (150083/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 12s, execs: 1728705 (147363/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 15s, execs: 2177603 (149645/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 18s, execs: 2622127 (147865/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 21s, execs: 3040752 (139063/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 24s, execs: 3475335 (145215/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 27s, execs: 3920852 (148601/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 30s, execs: 4375066 (151537/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 33s, execs: 4828779 (150568/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 36s, execs: 5254654 (142259/sec), new interesting: 0 (total: 34)
```

```

fuzz: elapsed: 39s, execs: 5679249 (142016/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 42s, execs: 6072884 (131266/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 45s, execs: 6462349 (129473/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 48s, execs: 6891584 (143024/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 51s, execs: 7141822 (83356/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 54s, execs: 7473331 (110793/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 57s, execs: 7826421 (117455/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 1m0s, execs: 8219327 (131249/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 1m0s, execs: 8219327 (0/sec), new interesting: 0 (total: 34)
PASS
ok      Fuzzing_Go_CSV 60.606s

```

```

fuzz: elapsed: 0s, gathering baseline coverage: 0/34 completed
fuzz: elapsed: 0s, gathering baseline coverage: 34/34 completed, now fuzzing with 8 workers
fuzz: elapsed: 3s, execs: 398470 (132417/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 6s, execs: 837395 (146362/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 9s, execs: 1287657 (150083/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 12s, execs: 1728705 (147363/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 15s, execs: 2177603 (149645/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 18s, execs: 2622127 (147865/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 21s, execs: 3040752 (139063/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 24s, execs: 3475335 (145215/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 27s, execs: 3920852 (148601/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 30s, execs: 4375066 (151537/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 33s, execs: 4828779 (150568/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 36s, execs: 5254654 (142259/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 39s, execs: 5679249 (142016/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 42s, execs: 6072884 (131266/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 45s, execs: 6462349 (129473/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 48s, execs: 6891584 (143024/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 51s, execs: 7141822 (83356/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 54s, execs: 7473331 (110793/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 57s, execs: 7826421 (117455/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 1m0s, execs: 8219327 (131249/sec), new interesting: 0 (total: 34)
fuzz: elapsed: 1m0s, execs: 8219327 (0/sec), new interesting: 0 (total: 34)
PASS
ok      Fuzzing_Go_CSV 60.606s

```

图 3 在 Go 中验证和存储 CSV 数据模糊测试结果截图

模糊测试运行了 60 秒，执行了超过 8,210,000 次函数调用，并发现了 0 个新的“interesting”输入，使得总数达到了 34 个。

（三）实验结论

1、模糊测试的有效性

本实验展示了模糊测试在发现边界情况和潜在漏洞中的有效性。通过大量的自动化输入生成，模糊测试能够快速且高效地覆盖多种输入场景，为检测异常行为和验证函数正确性提供了保障。

2、模糊测试的局限性

尽管模糊测试能发现边界情况，但它依赖于种子输入和预定义的验证规则。测试结果显示，字符串反转函数发现了 3 个新的“interesting”输入，而 CSV

验证函数则没有发现新的“interesting”输入，这可能是由于测试种子或验证逻辑对 CSV 数据格式约束较强。

（四）实验心得

1、模糊测试的实用性

模糊测试是验证软件质量的一种高效手段。通过生成大量随机或边界输入，能够有效检测代码在处理异常情况时的表现。在这次实验中，通过对 `ReverseString` 函数的测试，我体会到了模糊测试在发现潜在缺陷方面的优势。尤其是在处理多字节 Unicode 字符时，模糊测试能够揭示出常规测试可能遗漏的问题。

2、代码设计的重要性

在编写和测试代码时，良好的设计和清晰的函数接口至关重要。`ReverseString` 函数通过使用 `rune` 切片来正确处理 Unicode 字符，避免了因字符编码不当导致的潜在错误。此外，`validateAndSaveData` 函数的设计使其能够处理多种输入源，这种灵活性在测试中显得尤为重要。优秀的代码设计使得模糊测试能够更顺利地进行，减少了后期维护和测试的复杂性。

3、对边界情况的重视

模糊测试过程中，许多新发现的“interesting”输入强调了边界情况的重要性。在对 `validateRecord` 函数的测试中，尽管没有发现新的输入，但这也反映了函数的稳定性。同时，我意识到需要更加关注输入数据的各种组合，特别是在 CSV 数据验证中，输入的灵活性可能会影响到验证逻辑的适用性。

4、未来的改进方向

在未来的学习中，我计划进一步探索模糊测试的高级技术，例如使用更复杂的种子输入和结合其他测试方法，以增强测试的全面性和有效性。此外，还希望能将模糊测试与静态分析工具结合起来，以实现代码质量的全面提升。

三、关于模糊测试技术的思考与创新性观点

模糊测试技术作为软件安全领域的重要工具，其不断发展为漏洞检测和软件可靠性提供了强有力的保障。在阅读相关文献和进行简单的模糊测试实践之后，我有以下观点：

（一）融合人工智能与机器学习

模糊测试的未来发展可以与人工智能和机器学习相结合，以提高测试效率和漏洞发现率。例如，通过深度学习算法分析历史测试数据，识别出最常见的漏洞类型，从而在生成测试用例时优先考虑这些潜在弱点。这种基于数据驱动的智能模糊测试不仅能自动化生成测试用例，还能通过模型不断学习和优化，使得测试工具在针对特定软件时能够更加智能和高效。

（二）基于形式化方法的模糊测试

将形式化方法引入模糊测试，可以在一定程度上解决模糊测试中存在的路径爆炸问题。通过使用模型检测技术和符号执行方法，可以对输入进行严格的数学建模，从而在生成模糊测试用例时提供更可靠的路径约束。这种方法不仅可以提高测试的覆盖率，还可以确保在模糊测试过程中生成的测试用例能够涵盖特定的程序状态。

（三）增强动态反馈机制

在模糊测试的执行过程中，动态反馈机制至关重要。通过增强测试工具与被测程序之间的交互，可以实时监测程序的运行状态和异常行为，从而优化测试用例生成的策略。例如，采用实时监控和反馈算法，根据程序的实际运行情况动态调整测试用例的生成策略。这不仅能提高测试效率，还能降低误报率，从而使模糊测试更为精准。

（四）提升对未知协议和系统的适应性

随着互联网技术的发展，越来越多的未知协议和系统不断涌现，传统的模糊测试方法往往难以适应这种复杂性。创新性的模糊测试工具可以结合自动化逆向

工程技术，自动学习和适应新协议的特征，从而生成高效的模糊测试用例。这种自适应能力使得模糊测试能够在快速变化的技术环境中持续有效。

（五）实现跨平台的模糊测试框架

随着云计算和虚拟化技术的普及，跨平台的模糊测试框架将成为未来的发展趋势。通过设计支持多种编程语言和操作系统的模糊测试工具，开发者可以在不同的环境中灵活应用模糊测试。这种统一的平台不仅可以降低开发和维护成本，还能实现资源的共享与复用。

（六）社区协作与开源创新

推动模糊测试技术的开源化和社区协作，可以加速技术的发展和 innovation。通过建立开放的模糊测试框架和共享的种子库，开发者可以共同研究和改进模糊测试算法和工具。这种开放的合作方式将促进知识的共享和技术的进步，从而提升整个行业的安全水平。

参考文献

- [1] Barton P. Miller, Lars Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Commun. ACM* 33, 12 (Dec. 1990), 32–44.
- [2] 汪美琴, 夏旻, 贾琼, 等. 模糊测试技术的研究进展与挑战[J]. 信息安全研究, 2024,10(07):668-674.
- [3] 季王鑫. 面向漏洞挖掘的模糊测试优化方法研究[D]. 吉林化工学院, 2024.DOI:10.27911/d.cnki.ghjgx.2024.000186.
- [4] 李伟明, 郭瑾仪, 唐娜. 基于程序控制流图的模糊测试漏洞挖掘方法[J]. 武汉大学学报(工学版), 2023,56(09):1146-1153.DOI:10.14188/j.1671-8844.2023-09-015.
- [5] 任泽众, 郑晗, 张嘉元, 等. 模糊测试技术综述[J]. 计算机研究与发展, 2021,58(5):944-963.DOI:10.7544/issn1000-1239.2021.20201018.
- [6] 牛胜杰, 李鹏, 张玉杰. 模糊测试技术研究综述[J]. 计算机工程与科学, 2022,44(12):2173-2186.DOI:10.3969/j.issn.1007-130X.2022.12.011.
- [7] 张子涵, 赖清楠, 周昌令. 深度学习框架模糊测试研究综述[J]. 信息网络安全, 2024, 24(10):1528-1536.
- [8] <https://www.51cto.com/article/799766.html>

致 谢

在此，我要衷心地感谢张淼老师在《软件安全》课程中所教授的知识，让我对软件安全领域有了更为深刻的认识与理解。感谢老师和助教们在课后辛勤地付出，批改作业以及回答问题。感谢老师和同学们的指导与支持，让我在学习的过程中能够收获丰富的见解和灵感。同时，我也要感谢我的家人对我学业的支持与鼓励，让我能够专注于研究和探索。“路漫漫其修远兮吾将上下而求索”，在软件安全这个乐趣与挑战并存的领域里，我将继续努力前行。